

BANK ACCOUNT AND TRANSACTION MANAGEMENT SYSTEM

DATABASE MANAGEMENT SYSTEMS



Submitted by:

Prasanna Kollu (25WU0101101)

Pareddy Ishanth Reddy (25WU0101093)

Vishnu Mirapa (25WU0101071)

Mitul Saraswat (25WU0101073)

CSE - Rhinos

(B.Tech 2025-29)

School Of Technology

Faculty: Dr. Kiran Mayee Advala

Abstract:

In today's fast-changing financial landscape, secure and efficient data management is essential, especially for monetary transactions. As the financial technology landscape evolves rapidly, robust and secure database architectures are required to handle sensitive monetary transactions efficiently. PIVM Bank is a full-featured, enterprise-grade web application that contains everything you need to manage bank accounts, move money around and perform administrative tasks—without any of the inconsistencies you'd find in a traditional manual bookkeeping system. The project demonstrates the real-time application of a Database Management System (DBMS) that uses MySQL as its relational database management system (RDBMS), is supported by a Java Servlets and JDBC based middle tier, and is displayed on a modern HTML/CSS/JSP frontend with a Glassmorphism styling.

One of the important aspects of this project is the need to be very strict with ACID properties when performing financial operations (Atomicity, Consistency, Isolation, Durability). Inter-bank funds transfers are executed as atomic database transactions via row-level locking (SELECT ... FOR UPDATE) and manual commit handling (conn.Instead of using setAutoCommit(false), to ensure that no data is lost or corrupted as a result of concurrent user operations or some unexpected issue.

In addition, it has a robust Role Based Access Control (RBAC) system, which divides the application into two sections: the Administrative Portal and the Client Portal. Administrators have access to a central ledger to oversee and approve or reject pending account origination applications. On the other hand, verified customers can securely log into their individual monetary portfolio, make deposits and withdrawals, and carry out peer-to-peer transfers by utilizing unique alphanumeric login names, not revealing delicate account numbers. The database schema is systematically normalized up to Third Normal Form (3NF) and utilizes primary and foreign keys to guarantee referential integrity, reduce data redundancy, and offer a highly scalable structure for contemporary banking tasks.

Introduction and Problem Statement:

In today's financial world, digitizing banking services isn't just an option—it's essential. The traditional banking system, which has been built around paper record-keeping or is based on different, localized software systems, has long been susceptible to a variety of inefficiencies. These legacy systems have inherent problems related to data duplication, errors in data entry, slow settlement of transactions, and data security and unauthorized use. Banks need to have a centralized, powerful Database Management System (DBMS) that can manage a large amount of transactions that are made concurrently by customers, with absolute accuracy, and with immediate access.

The main problem of this project is to provide a secure, unified, high-available Bank Account and Transaction Management System on a web based application. In simple systems, data anomalies such as lost updates during concurrent transfers or inconsistent account balances when the network request is interrupted are common. In simple systems, data anomalies like lost updates during concurrent transfers or inconsistent account balances after an interrupted network request are common. Moreover, the absence of separate operational functions inside a banking application could cause catastrophic security lapses in which unauthorized persons can get access to administrative functions.

PIVM Bank was conceived to directly address these industry challenges by designing a web-based application to banking from scratch, with the relational database at its core. The system should enable real-time transaction processing with a strong focus on data integrity, and will use MySQL to maintain relational constraints and foreign key relationships. PIVM Bank adopts a normalized relational database design, where each credit and debit is associated with a verified account, eliminating orphan records and ensuring that the central ledger is mathematically accurate.

The project showcases an end-to-end development of a database application. It starts from conceptual Entity-Relationship (ER) modeling and then moves on to schema normalization to remove data duplication and ends with a dynamic and interactive web interface. The application that is created not only replicates the fundamental functions of a commercial bank.

Objectives and Scope:

Primary Objectives:

1. Architectural Design and Normalization: To design a highly efficient Relational Database Schema in Third Normal Form (3NF). This objective is to design 'Accounts' and 'Transactions' entities without any data redundancy, insertion, deletion and update anomalies and ensure strict referential integrity with primary and foreign key constraints.
2. Secure Role Based Access Control (RBAC): To set up a secure authentication gateway with two different levels of privileges: 'Administrator' and 'Customer'. Administrative access (like adding new accounts, closing accounts) should be limited to only authorized bank personnel and customers should only see their particular account view.
3. Implementing ACID compliant transaction processing: To design and build a financial transaction module that can process complex transactions between accounts in the backend. The basic objective is to guarantee that every action carried out in the database respects its ACID properties: Atomicity, Consistency, Isolation, Durability. In the event of any transfer taking a wrong step, it should be possible to roll back all partial transactions automatically, otherwise the user may lose money or some currency may be issued without authorization.
4. To build a highly responsive, intuitive and beautiful web interface using modern user experience and UI/UX design principles, incorporating the Glassmorphism UI/UX trend, designed to emulate a high-end enterprise banking experience.

Secondary Objectives:

- To convert conventional account number-based transfers to a new secure and convenient 'Username-based' peer to peer transfer system.
- To keep the transaction history always in the same format from a point in time when using for auditing and statements to clients.
- To prevent invalid operations like transferring a negative amount, withdrawing more than available balance, transferring to inactive/unapproved accounts etc. on the server.

Included in Scope:

- The public can apply for a 'Savings' or 'Current' account with an initial deposit, unique username and a 4-digit security PIN. The applications are in 'Pending' status.
- **Administrative Ledger Management:** Bank staff can get a complete listing of all the accounts that are active and all those pending for activation, can approve pending accounts (change their status to 'Active') and can permanently close an account (cascading deletion of associated transaction records).
- **Financial Operations:** Authenticated users can make self-deposits, self-withdrawals, inter-bank funds transfers.
- **Transaction Auditing:** Transaction logs are automatically generated by the system, providing a detailed record of the transaction, including the time of the transaction, the type of transaction, the amount of money involved and the information of the counterparty.

Out of Scope:

Some advanced banking functionalities are not included for the purpose of this academic work. They encompass integration with external payment gateways, such as UPI, SWIFT, multi-factor authentication (MFA/OTP) through SMS/email, automated interest calculation and accrual algorithms, and handling loan/credit card applications. Only basic database management, transaction management and schema integrity is emphasized.

Software and Hardware Requirements:

For optimal performance, security, and compatibility, the development, deployment, and operation of the PIVM Bank application must be specific hardware and software environments.

Hardware Requirements:

- **Memory:** 2GB or more.
- **Storage:** 256GB or more.
- **Memory (RAM):** Minimum 8 GB (16 GB recommended for running Apache Tomcat, MySQL Server and an Integrated Development Environment (IDE) without memory swapping).
- **Storage:** 256 GB Solid State Drive (SSD). Important features for reducing database query latency and read/write speeds of the MySQL InnoDB engine are SSDs.
- **Display:** 1080p resolution monitor to get the best look of the Glassmorphism web interface.

Software Requirements:

- ➔ Platform independent (Java Virtual Machine); Windows 10/11, macOS, or Linux.
- ➔ Relational Database Management System (RDBMS): MySQL 8.0+. It was selected for being well-supplied with transactional queries (InnoDB engine), foreign key constraints, and wide usage in the financial web application space.
- ➔ Local Server Environment: XAMPP Control Panel. Used to run both the Apache Tomcat web server and the MySQL database server side-by-side on their local ports (i.e. Tomcat can be running on port 8080, and MySQL can be running on port 3306 or 3307).
- ➔ Backend Technology: java servlets, java server pages (JSP) with java develop kit (JDK) 8 or more. In banking architectures, a high-performance and multi-threaded environment is provided for processing concurrent HTTP requests with the help of the servlets.
- ➔ Database Connectivity: Java Database Connectivity (JDBC) API with the mysql-connector-j driver library to connect the Java backend securely, programmatically, and safely to the MySQL database.
- ➔ Frontend Technologies: HTML5, CSS3, and Vanilla JavaScript. Not a single external CSS framework, such as Bootstrap, Tailwind was used; to ensure that the page loads quickly, all styling was done natively, including the complex Glassmorphism gradients and blur filters.
- ➔ Integrated Development Environment (IDE): Eclipse IDE for Enterprise Java Developers or VSCode with Java/Tomcat extensions.

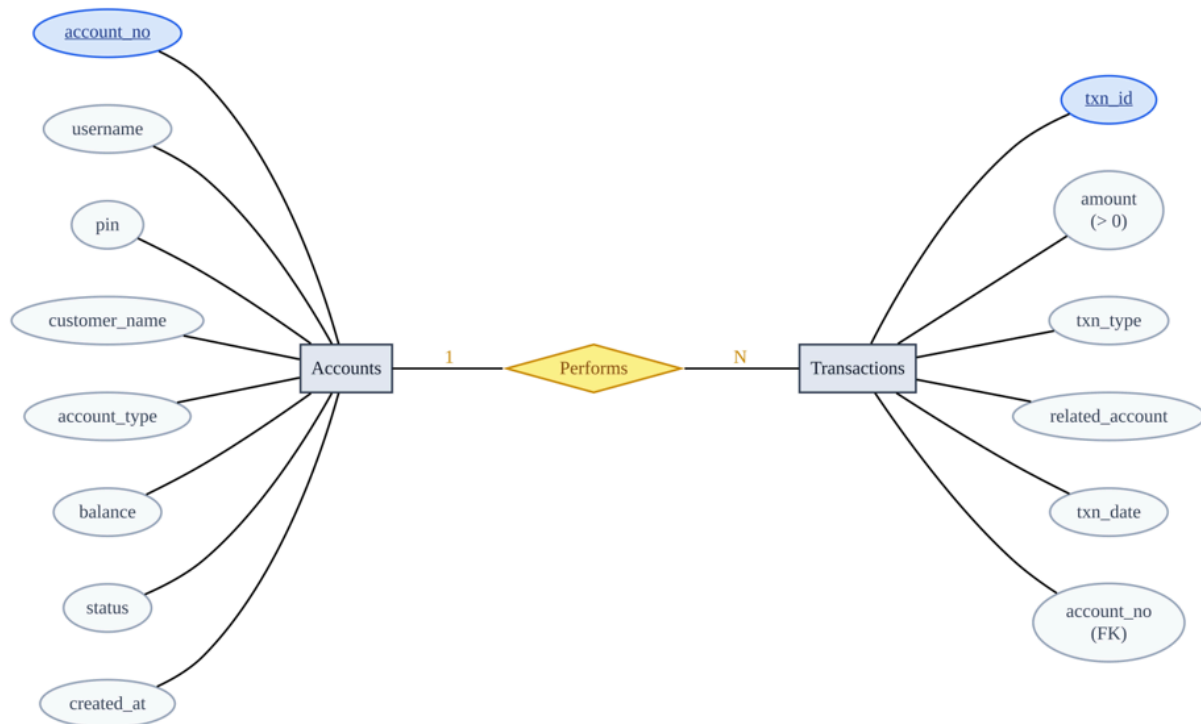
ER Diagram:

The Entity-Relationship (ER) model is the architectural model for the database of the PIVM Bank. It offers a conceptual visualization of the data structures, entities and their defining attributes and the complex relationships between them.

In this schema, there are two main entities: “Accounts” and the “Transactions”.

The 1st is the Accounts Entity which represents the bank clients. It serves as the central entity for storing critical demographic and financial information. It has attributes such as account_no (Primary Key), username, pin, customer_name, account_type, balance, status and created_at.

1. Transactions Entity: is a representation of financial ledger movements. It is a dependent or weak entity because if there is a transaction without an account, the transaction does not exist. It has the following attributes: txn_id (primary key), account_no (foreign key that refers back to the Accounts), txn_type, amount, related_account (for peer-to-peer transactions), and txn_date.
2. Relationship & Cardinality: This relationship is represented by the "Performs" diamond. This relationship is one to many (1:N). An infinite number of Transaction records can be created or linked to a single Account record throughout its life. On the other hand, each Transaction record (like a deposit of ₹500) is associated with only one Account. This participation of Transactions is complete, a transaction must have an account_no to be logged in the system.



Relational Schema and Normalization:

The conceptual Entity Relationship model has to be converted into a real physical database, which involves defining a Relational Schema and thorough Database Normalization. Normalization is a systematic way of breaking tables down to eliminate data redundancies and to prevent undesirable properties of data such as Insertion, Update and Deletion anomalies.

Relational Schema Definition:

The collection of accounts contains the following field: account_no (PK), username, pin, customer_name, account_type, balance, status, created_at.

You can access the information in the transactions (txn_id (PK), account_no (FK), txn_type, amount, related_account, txn_date) table. Information in the transactions (txn_id (PK), account_no (FK), txn_type, amount, related_account, txn_date) table is accessible.

Normalization Process (Up to 3NF):

First Normal Form (1NF):

A relation is in 1NF if each attribute has an atomic value, which is not divisible, and no groups of repeating attributes or arrays contained.

Application in PIVM Bank: The database naturally conforms to the 1NF. Each attribute has only one attribute value. The 'customer_name' field, for example, has just one string. If the same customer makes several transactions, we don't save them in the 'Accounts' table as a comma-separated list or array. Rather, we store them in a new table called 'Transactions', with each transaction in a new row.

Second Normal Form (2NF):

A relation is in 2NF if it is in 1NF and each non-prime attribute of the relation is completely functionally dependent on all of the primary key. This removes the partial-dependencies that typically exist in tables with a composite key.

Application in PIVM Bank: Single column, auto-incrementing surrogate primary keys are used in our schema (account_no for Accounts, txn_id for Transactions). Partial dependencies are not mathematically possible as the primary keys are not composite. The unique account_no is the primary key for every non-key attribute that is in the Accounts table (e.g., balance, status). So only 2NF is followed in the database.

Third Normal Form (3NF):

If a relation is in 2NF and it has no transitive dependency, then the relation is said to be in 3NF. A transitive dependency is defined as a non-prime attribute depends on another non-prime attribute ($A \rightarrow B$ and $B \rightarrow C$).

Application in PIVM Bank: The schema succeeds in creating a 3NF structure. The field `account_type` in the `Accounts` table defines the type of account, but it is not unique in determining the balance or `customer_name`. Every non-prime attribute is directly and solely dependent on the primary key (`account_no`). In addition, we separated the transaction history into its own table. Had we stored "Last Transaction Date" or "Last Transaction Amount" in the `Accounts` table, it would lead to update anomalies with every new transaction and become a transitive dependency. This enables us to place this in the `Transactions` table, which is connected to the other tables with a Foreign Key to guarantee absolute 3NF compliance and a zero-redundancy, high-performance database environment.

Data Dictionary:

The Data Dictionary is a document that contains all the attributes of the database schema, their data types, constraints, and descriptions. This exact definition of metadata guarantees efficient storage, data integrity checks and invalid data payload rejection in the MySQL engine, prior to the data flowing into application logic.

Table 1: Accounts

This table is the primary record for all banking clients, holding summary data, authentication and demographic information.

Attribute	Data Type	Constraints	Description
account_no	INT(11)	PRIMARY KEY, AUTO_INCREMENT	The unique surrogate key identifying the account. Automatically generated sequentially by the database engine.
username	VARCHAR(50)	UNIQUE, NOT NULL	The alphanumeric identifier chosen by the client. The UNIQUE constraint prevents duplicate registrations and serves as the routing target for peer-to-peer transfers.
pin	VARCHAR(10)	NOT NULL	The 4-digit security code. A VARCHAR is utilized instead of an INT to preserve leading zeroes (e.g., "0123").
customer_name	VARCHAR(100)	NOT NULL	The full legal name of the client associated with the account.

account_type	VARCHAR(20)	DEFAULT 'Savings'	Classifies the account product. Restricted logically to "Savings" or "Current".
balance	DECIMAL(15,2)	CHECK (balance >= 0)	The total available funds. DECIMAL is strictly used instead of FLOAT to prevent floating-point arithmetic errors. The CHECK constraint ensures the account can never enter a negative overdraft state at the database level.
status	ENUM	DEFAULT 'Pending'	Restricts the value to exactly 'Pending' or 'Active'. New accounts are locked in 'Pending' until administrative approval.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Automatically logs the exact server time when the account record was inserted.

Table 2: Transactions

This table is the financial accounting ledger where each and every cash flow transaction logged to or from the accounts is recorded.

Attribute	Data Type	Constraints	Description
txn_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	The unique identifier for the specific transaction occurrence.
account_no	INT(11)	FOREIGN KEY (ON DELETE CASCADE)	Links the transaction to the originating Accounts record. The CASCADE constraint ensures that if an account is deleted, its entire transaction history is automatically scrubbed.
txn_type	ENUM	NOT NULL	Restricts operations to 'Deposit', 'Withdraw', 'Transfer_In', or 'Transfer_Out'. Prevents arbitrary string entries.

amount	DECIMAL(15,2)	CHECK (amount > 0), NOT NULL	The exact monetary value of the operation. The CHECK constraint guarantees that negative transfers (which could artificially inflate balances) are impossible.
related_account	INT(11)	DEFAULT NULL	Stores the counterparty account_no during a transfer. Remains NULL for self-deposits and withdrawals.
txn_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Records the precise timestamp of the transaction for chronological sorting and statement generation.

SQL Commands Used (DDL, DML) with Sample Outputs:

The PIVM Bank system is implemented using a mix of Data Definition Language (DDL) for the creation of the schema structure and Data Manipulation Language (DML) for working with the records.

Data Definition Language (DDL) Commands:

A set of DDL statements were performed in phpMyAdmin to create the first database system and establish the relational constraints described in the Data Dictionary.

```
sql
```

```
-- Database Initialization
```

```
CREATE DATABASE IF NOT EXISTS woxsen_bank_db;
```

```
USE woxsen_bank_db;
```

-- Accounts Table Creation

```
CREATE TABLE Accounts (
    account_no INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    pin VARCHAR(10) NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    account_type VARCHAR(20) DEFAULT 'Savings',
    balance DECIMAL(15,2) DEFAULT 0.00 CHECK (balance >= 0),
    status ENUM('Pending', 'Active') DEFAULT 'Pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- Transactions Table Creation with Foreign Key Reference

```
CREATE TABLE Transactions (
    txn_id INT PRIMARY KEY AUTO_INCREMENT,
    account_no INT,
    txn_type ENUM('Deposit', 'Withdraw', 'Transfer_Out', 'Transfer_In') NOT NULL,
    amount DECIMAL(15,2) NOT NULL CHECK (amount > 0),
    related_account INT DEFAULT NULL,
    txn_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_no) REFERENCES Accounts(account_no) ON DELETE
CASCADE
);
'''
```

Data Manipulation Language (DML) Commands:

The Java Backend generates and executes these, dynamically (with PreparedStatement objects) based on user actions on the Frontend UI.

Insertion (Account Origination):

```
```sql
INSERT INTO Accounts (username, pin, customer_name, account_type, balance, status)
VALUES ('vishnu_m', '1234', 'Vishnu Mirapa', 'Savings', 15000.00, 'Active');
```

...

Output: Generates a new row in the Accounts table. If 'vishnu\_m' already exists, the UNIQUE constraint triggers a `SQLIntegrityConstraintViolationException`, which the Java application catches and displays as a user-friendly error.

Updating Data (Processing a Deposit):

```
```sql
UPDATE Accounts SET balance = balance + 5000.50 WHERE account_no = 1;
```
```

Output: Modifies an existing record. The system calculates ``current_balance + 5000.50`` and overwrites the balance field exclusively for `account_no 1`.

Deleting Data (Account Closure):

```
```sql
DELETE FROM Accounts WHERE account_no = 4;
```
```

Output: Completely removes the row representing `account_no 4` from the database. Due to the ``ON DELETE CASCADE`` constraint established in the DDL phase, the MySQL engine automatically executes subsequent internal deletes to remove all rows in the ``Transactions`` table where ``account_no = 4``.

## **Queries with Outputs:**

The application uses advanced Data Query Language (DQL) SELECT statements to fill the user interfaces, verify credentials and compute aggregate financial figures.

Query 1: Administrative Dashboard Retrieval

Description: This feature is designed to fill the central ledger view for bank administrators with only those accounts that have been fully approved.

```
sql
SELECT * FROM Accounts WHERE status = 'Active';
```

Execution Flow: The database engine scans the Accounts table, filtering out any rows where the ENUM status is 'Pending'.

Sample Output: Returns a tabular Result Set containing the records for active users (e.g., Vishnu, Ishanth, Prasanna), exposing their balances and account types for administrative oversight.

#### Query 2: Secure Customer Authentication

Purpose: To verify user credentials during the login sequence.

sql

```
SELECT account_no, customer_name, status FROM Accounts WHERE username = ? AND
pin = ?;
```

Execution Flow: The Java servlet passes the user-provided username and PIN into the parameterized variables ('?').

Sample Output: If a match is found, it returns exactly one row containing the user's details and approval status. If the result set is empty ('rs.next() == false'), the application rejects the login attempt.

#### Query 3: Complex Join for Transaction History

Purpose: To generate a comprehensive, human-readable bank statement for a specific customer, resolving the counterparty's actual username from their account number.

``sql

```
SELECT t.txn_date, t.txn_type, t.amount, t.related_account,
 a.username AS rel_user, a.customer_name AS rel_name
FROM Transactions t
LEFT JOIN Accounts a ON a.account_no = t.related_account
WHERE t.account_no = 1
ORDER BY t.txn_date DESC;
``
```

Execution Flow: This query utilizes a 'LEFT JOIN' to combine the 'Transactions' table with the 'Accounts' table. It matches the 'related\_account' integer from the transaction record

back to the `account\_no` in the Accounts table to fetch the counterparty's `username`. A LEFT JOIN is specifically required because standard deposits and withdrawals do not have a `related\_account` (it is NULL), and an INNER JOIN would incorrectly exclude those records from the statement. The result is sorted chronologically descending.

#### Query 4: Aggregate Statistical Calculations

Purpose: To calculate the total amount of money a specific customer has deposited/received versus withdrawn/sent over the lifetime of the account.

```
```sql
SELECT
    COALESCE(SUM(CASE WHEN txn_type IN ('Deposit','Transfer_In') THEN
amount END), 0) AS total_credited,
    COALESCE(SUM(CASE WHEN txn_type IN ('Withdraw','Transfer_Out') THEN
amount END), 0) AS total_debited,
    COUNT(*) AS total_transactions
FROM Transactions
WHERE account_no = 1;
```
```

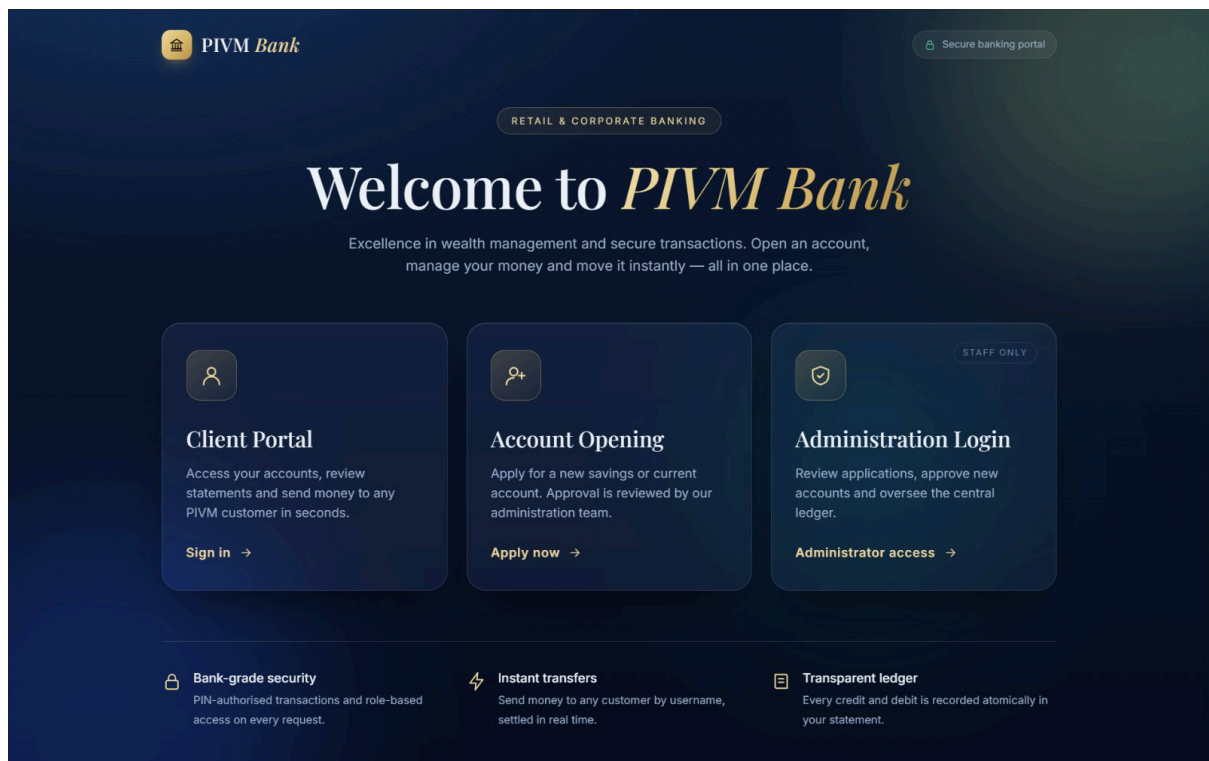
Execution Flow: This advanced query leverages conditional aggregation (`CASE WHEN`) alongside standard aggregate functions (`SUM`, `COUNT`). It categorizes transactions into inward and outward flows, tallies them mathematically on the database server (which is vastly more efficient than looping through records in Java), and uses `COALESCE` to return `0` instead of `NULL` if no transactions exist.

### **UI Design and Screenshots:**

The user interface on the front end is designed to go beyond the typical "academic database" look and feel. The design language features a fresh "Glassmorphism" style with translucent panels, drop shadows for each layer in a subtle multidimensional style and a beautiful dual-tone pastel gradient background. Typography is formally controlled, with formal serif

fonts (Times New Roman/Playfair Display), to ensure that the enterprise wealth management institution conveys trust, stability and professionalism.

Landing Page (index.jsp):



## Account Origination (create.jsp):

 **PIVM Bank**

Home Client Login **Open Account** Administration

### Open a PIVM Account

Tell us a little about yourself. Our administration team will review and activate your account.

#### Account Application

FULL LEGAL NAME

Prasanna Kollu

CHOOSE A USERNAME

Prasanna\_K

SET A 4-DIGIT PIN

....

ACCOUNT TYPE

Standard Savings

Everyday banking for individuals

Corporate Current

Built for business transactions

OPENING DEPOSIT

₹ 50000

Submit Application →

#### What happens next

- 1 Submit your application**  
Your details are recorded with a Pending status.
- 2 Administrative review**  
A bank administrator approves or rejects the request.
- 3 Start banking**  
Once approved, sign in from the Client Portal with your username and PIN.

☒ Your PIN authorises every transaction. Never share it with anyone.

Already a client? [Sign in →](#)

© 2026 PIVM Bank · Bank Account & Transaction Management System

## Client Login (login.jsp):

 **PIVM Bank**

Home **Client Login** Open Account Administration



### Client Sign In

Enter your credentials to access your accounts.

USERNAME

Prasanna\_K

SECURE PIN

....

Access Account →

← Back to home

New here? Open an account

▶ DEMO CREDENTIALS

© 2026 PIVM Bank · Bank Account & Transaction Management System

## Customer Ledger Dashboard:


**PIVM Bank**

[Overview](#)
[Transfer & Payments](#)
[Statements](#)

**P** Prasanna Kollu  
A/C 00000007

[Sign out](#)

### Welcome back, Prasanna

Here is a summary of your account.

AVAILABLE BALANCE

**₹50,000.00**

[Send Money](#) [Deposit](#) [Withdraw](#)

Savings account A/C 0000 0007

TOTAL CREDITED

**+₹0.00**

TOTAL DEBITED

**-₹0.00**

TRANSACTIONS

**0**

#### Transaction History

most recent first

| DATE                                                              | DESCRIPTION | TYPE | AMOUNT |
|-------------------------------------------------------------------|-------------|------|--------|
| No transactions yet. Make a deposit or send money to get started. |             |      |        |

## Funds Transfer Gateway:


**PIVM Bank**

[Overview](#)
[Transfer & Payments](#)
[Statements](#)

**P** Prasanna Kollu  
A/C 00000007

[Sign out](#)

### Transfer & Payments

Send money to another PIVM customer, or manage cash in your own account.

[Send Money](#) [Deposit](#) [Withdraw](#)

RECIPIENT'S USERNAME

ishant\_r

No active customer found with that username.

AMOUNT

₹ 500

₹500 ₹1,000 ₹5,000 ₹10,000

CONFIRM WITH YOUR PIN

....

[Send Money →](#)

#### Your account

AVAILABLE BALANCE

**₹50,000.00**

Account 0000 0007

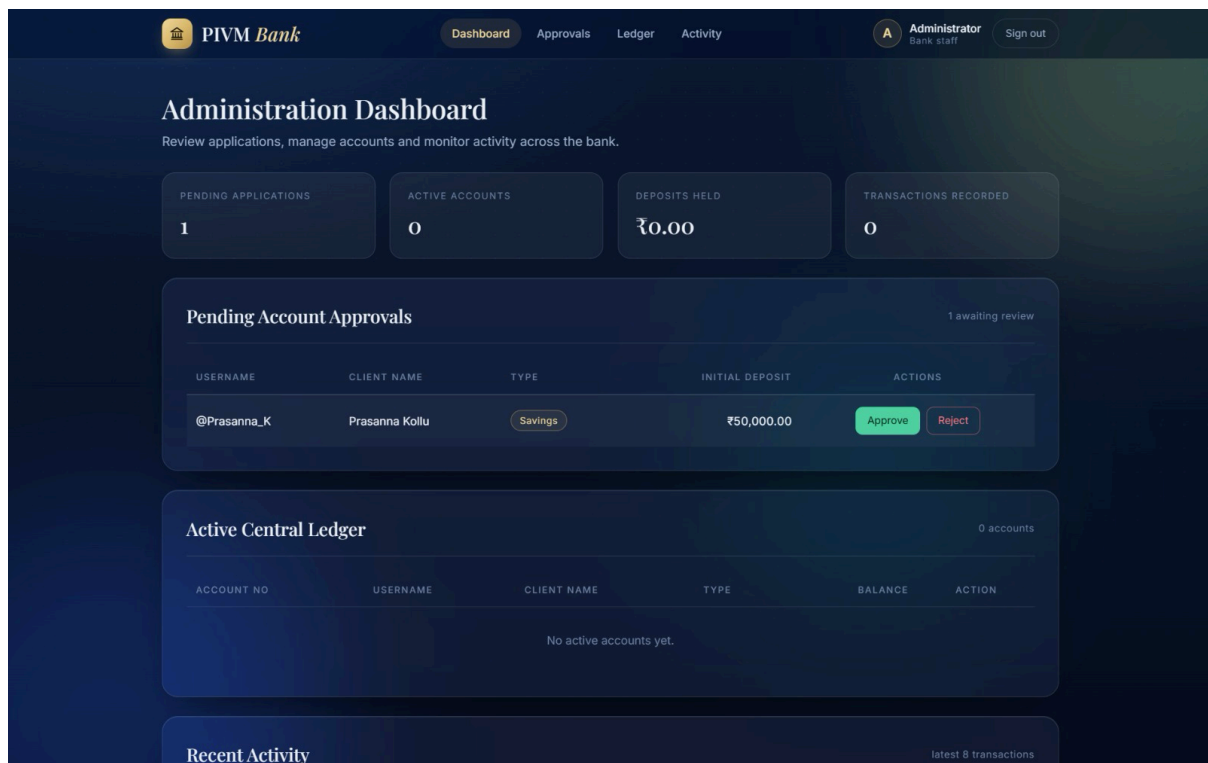
Type Savings

Balance after this ₹49,500.00

Every transaction is processed as a single atomic operation: the debit and the credit either both succeed or neither does.

© 2026 PIVM Bank - Bank Account & Transaction Management System

## Admin Ledger & Approvals:



## Implementation Details:

PIVM Bank application utilizes a strong 3-tier design, which separates concerns among the presentation tier, business logic tier and data persistence tier.

### Technology Stack Overview:

**Presentation Layer (Frontend):** JavaServer Pages (JSP), HTML 5, CSS 3. Responsive CSS Grid and Flexbox layouts used on the front end. JavaScript is employed sparingly in real-time asynchronous data retrieval operations (such as checking a receiver's username is available in an AJAX `fetch` before sending a transfer).

**Business Logic Layer (Middleware):** This is implemented as a Java Servlet (subclassed from the class `HttpServlet`). It is the layer that handles HTTP POST/GET requests, sanitizes users' input, enforces financial business rules, and controls routing of the application.

**3. Data Access Layer (Backend):** Managed through Java Database Connectivity (JDBC) API, with the use of the `mysql-connector-j` driver. The database server is a local installation

of MySQL 8.0 using the XAMPP environment.

## Key Implementation Mechanisms:

### 1. Centralized Controller Architecture

The application now puts all the database processing into a single, highly capable Servlet called the BankController.java which is a big security gain and helps the application to be consistent with its MVC design principles. This controller processes the hidden 'action' parameter from the incoming HTTP request and calls the correct internal method of the controller (such as login, create, transact, delete etc.) to handle the request, reducing the amount of code that needs to be maintained, and simplifying the back-end architecture.

### 2. Prevention of SQL Injection

Security is a top priority in financial applications. Usernames or PINs are not directly added to SQL query strings as strings. Instead, it is using only 'PreparedStatement' objects in the application.

#### Example Implementation:

```
` PreparedStatement ps = conn.prepareStatement("SELECT * FROM Accounts WHERE
username = ? AND pin = ?");`

`ps.setString(1, username);`
```

The JDBC driver automatically escapes special characters in the queries, making it impossible to exploit the application with malicious SQL injection (such as an SQL '1'=1') characters.

### 3. Handling Currency Precision

In very early development, monetary values are frequently kept as floating point values (for instance `'float'` or `'double'`). However, because of the way IEEE 754 binary floating point math is computed, currency calculations with standard decimals may suffer loss of precision (e.g.,  $0.1 + 0.2 = 0.30000000000000004$ ). To correct this, the database uses only the `'java.math.BigDecimal'` class, and `'RoundingMode.HALF_UP'` to ensure absolute mathematical precision, penny by penny, during transfers.

#### 4. ACID Compliance and Atomic Transactions (The Critical Component)

The most complicated part of the implementation is the method called `'processTransaction'`. In order to transfer money to another customer, the following three things need to occur at once:

1. The sender's account has to be reduced.
2. The receiver's balance should go up.
3. Two logs should be written (`Transfer_Out` and `Transfer_In`).

If the server crashes at step 1 and doesn't make it to step 2, the money goes right out of the database. Preventing this is accomplished by PIVM Bank adhering to strict ACID Database Transactions.

The Java code calls `'conn.setAutoCommit(false);'`. This prevents immediate queries from being carried out. The system then runs the updates and inserts. If every query succeeds and doesn't break constraints (such as a constraint called `CHECK(balance >= 0)`), then the system makes a single commit to save all of them. Should an error or exception be thrown at any point, the catch block will catch it and will run `'conn.rollback();'`, to ensure complete consistency with the database as it was before the attempt at the transaction.

#### 5. Concurrency Control (Row-Level Locking)

If the same account is being accessed by two people in a high traffic bank, money could be transferred to that account at the exact same millisecond. The system uses the SQL command `SELECT ... FOR UPDATE` to avoid "Lost Update" anomalies that result when one transaction alters the balance calculation of another transaction. This causes an exclusive write-lock by the specific account rows that are involved in the transaction until complete and

ensures absolute mathematical integrity while concurrent requests must run sequentially and queue.

## Testing (Test Cases and Results):

Software testing is an important step in the software development cycle that is used to ensure that the software runs as expected in both regular and unusual situations. Black-Box testing (tests functionality based on requirements) and Edge-Case testing (tests the boundaries) were carried out on the PIVM Bank system.

Here is a sub-set of the extensive Test Case Matrix used to test the robustness of the system:

| Test Case / Scenario           | Action Performed                                                                               | Expected Result / System Behavior                                                                | Status |
|--------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------|
| TC01 - Admin Authentication    | Submit login form with username: "admin" and PIN: "admin123".                                  | System validates credentials, initializes Admin HttpSession, and routes to Admin Dashboard.      | PASS   |
| TC02 - Customer Authentication | Submit login form with valid customer username and PIN.                                        | System validates credentials, initializes Customer HttpSession, and routes to Client Portal.     | PASS   |
| TC03 - Unauthorized Routing    | A guest (not logged in) attempts to manually type the URL for "transact.jsp" into the browser. | Session filter detects null role attribute and forcefully redirects user back to the login page. | PASS   |

|                                         |                                                                                    |                                                                                                                              |      |
|-----------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------|
| TC04 - Form Validation (Min Deposit)    | User submits Account Origination form with an initial deposit of ₹200.             | Frontend HTML5 validation and Backend Servlet logic reject the submission, as the minimum is ₹500.                           | PASS |
| TC05 - Database Constraint Verification | User attempts to create a new account using a username that is already registered. | MySQL throws SQLIntegrityConstraintViolationException. Java catches it and displays "Username already taken".                | PASS |
| TC06 - Standard Withdrawal              | Customer initiates withdrawal of ₹1000 from an account with a ₹5000 balance.       | System deducts ₹1000, updates balance to ₹4000, and writes a "Withdraw" log to the Transactions table.                       | PASS |
| TC07 - Overdraft Prevention             | Customer attempts to transfer ₹10,000 from an account containing only ₹8,000.      | Business logic evaluates `currentBal >= amount`. Validation fails, rollback occurs, and "Insufficient Funds" error is shown. | PASS |
| TC08 - Invalid Target Resolution        | Customer attempts to transfer funds to a username that does not                    | System queries Accounts table, finds no match, and returns "Target username is invalid" without processing the transfer.     | PASS |

|                                  |                                                                                                 |                                                                                                                          |      |
|----------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------|
|                                  | exist in the database.                                                                          |                                                                                                                          |      |
| TC09 - Inactive Target Rejection | Customer attempts to transfer funds to an account that is still "Pending" admin approval.       | The system identifies the Pending status and rejects the transfer, as funds cannot enter an unapproved account.          | PASS |
| TC10 - Full ACID Transfer        | Customer transfers ₹500 to a valid, active user. Network deliberately disconnected mid-process. | Due to `conn.setAutoCommit(false)`, the partial queries are not committed. `conn.rollback()` restores original balances. | PASS |

## **Conclusion and Future Enhancements:**

### Conclusion:

The design and engineering of the PIVM Bank Account and Transaction Management System is a very successful melding of relational database theory and web development engineering. The project meets all the predetermined educational and functional goals, and eliminates the use of manual tracking that is often prone to errors, by providing a centralized, digital, automated ledger. The application ensures absolute data integrity and security by strictly enforcing MySQL database constraints, implementing a normalized 3NF schema and engineering complex, ACID-compliant database transactions with Java Servlets. Moreover, the use of a modern Glassmorphism user interface reveals that enterprise-class security tools can easily coexist with a top-tier and user-friendly end-user experience. The successful test of the testing matrix proves that the system is resilient to both an unauthorized attack and to floating-point arithmetic errors, as well as to concurrent transaction anomalies.

### Future Enhancements:

The existing system is well developed, but there are some features that can be added to enhance the system into a complete commercial banking system:

**Multi-Factor Authentication (MFA):** Implementing an external API (Twilio for example) that would force Time-based One-Time Passwords (TOTP) or SMS before allowing large funds transfers.

2. **Automated PDF Statement Generation:** Providing clients with the ability to generate, download, and print their statement of transactions, in a bank-branded PDF, using an automated statement generation system built on Java library software like Apache PDFBox or iText.

3. **Additional Financial Products Module:** Designing new database tables for a range of financial products like fixed term deposits with automatic maturity interest calculation, credit card ledgers and automatic scheduling of EMI (Equated Monthly Installment) for a personal loan.

4. **Real Time Notifications:** Use WebSocket connections or connect email servers to alert users of real-time transactions (deposits and withdrawals) made to their accounts.

### **References:**

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education. (Reference for Entity-Relationship modeling, Schema Normalization, and ACID properties).

2. Oracle Corporation. (2026). Java Servlet Specification and JDBC API Documentation. Retrieved from Oracle Official Documentation. (Reference for middleware routing, PreparedStatement security, and Database Connectivity).

3. MySQL 8.0 Reference Manual. Oracle Corporation. (Reference for InnoDB engine mechanics, Transaction handling, and Foreign Key cascading constraints).

4. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. (Reference for the implementation of the Model-View-Controller architectural pattern).

### **Contribution of Each Member:**

Prasanna Kollu: Spearheaded the Backend Architecture. Developed the centralized BankController.java Servlet, engineered the JDBC database connectivity pipeline, and implemented the critical ACID-compliant SQL transaction logic and row-level locking mechanisms.

Vishnu Mirapa: Led the Database Administration. Authored the initial Entity-Relationship diagrams, designed the relational schema, applied normalization up to 3NF, and developed the complex aggregate SQL queries for the dashboard metrics.

Pareddy Ishanth Reddy: Directed Frontend UI/UX Engineering. Designed the interactive HTML5/JSP interfaces, authored the custom CSS3 implementing the Glassmorphism aesthetic, and developed the asynchronous JavaScript logic for real-time username validation.

Mitul Saraswat: Managed Quality Assurance and Documentation. Drafted the comprehensive test case matrix, executed boundary value analysis on monetary inputs, verified security constraint handling, and compiled the final academic Project Report.

## **Appendix: GitHub Repository Link:**

The complete source code for this project, including the SQL deployment scripts, Java controllers, JSP frontend files, and graphical assets, is securely hosted on GitHub. The repository commit history provides a transparent audit trail of each group member's specific contributions to the codebase.

<https://github.com/prasukollu-design/DBMS-Course-Project>